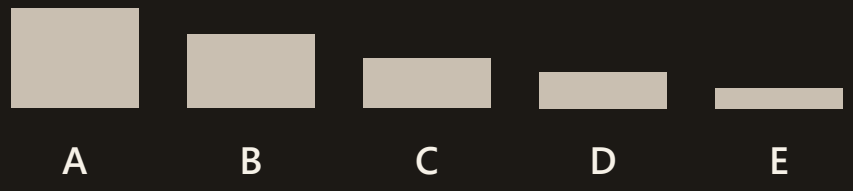
Sparse Attention as Compact Kernel Regression

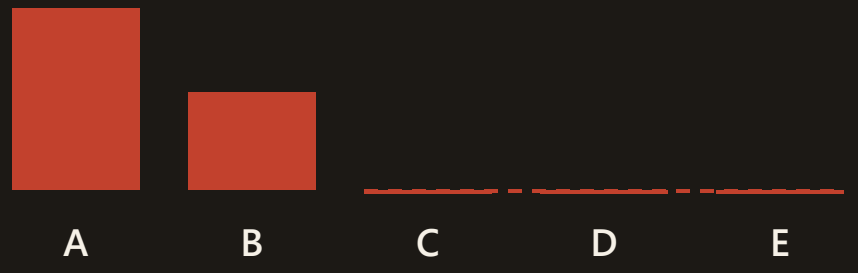
Разреженное внимание как ядерная регрессия

Почему точные нули меняют извлечение из памяти.

softmax: все ключи участвуют



sparsemax: часть весов равна нулю



Внимание извлекает ответ из памяти

Контекст содержит много записей. Лишь некоторые из них полезны для текущего запроса.

В памяти контекста

Лена	**чай**
Миша	**кофе**
Оля	**вода**

Новый запрос

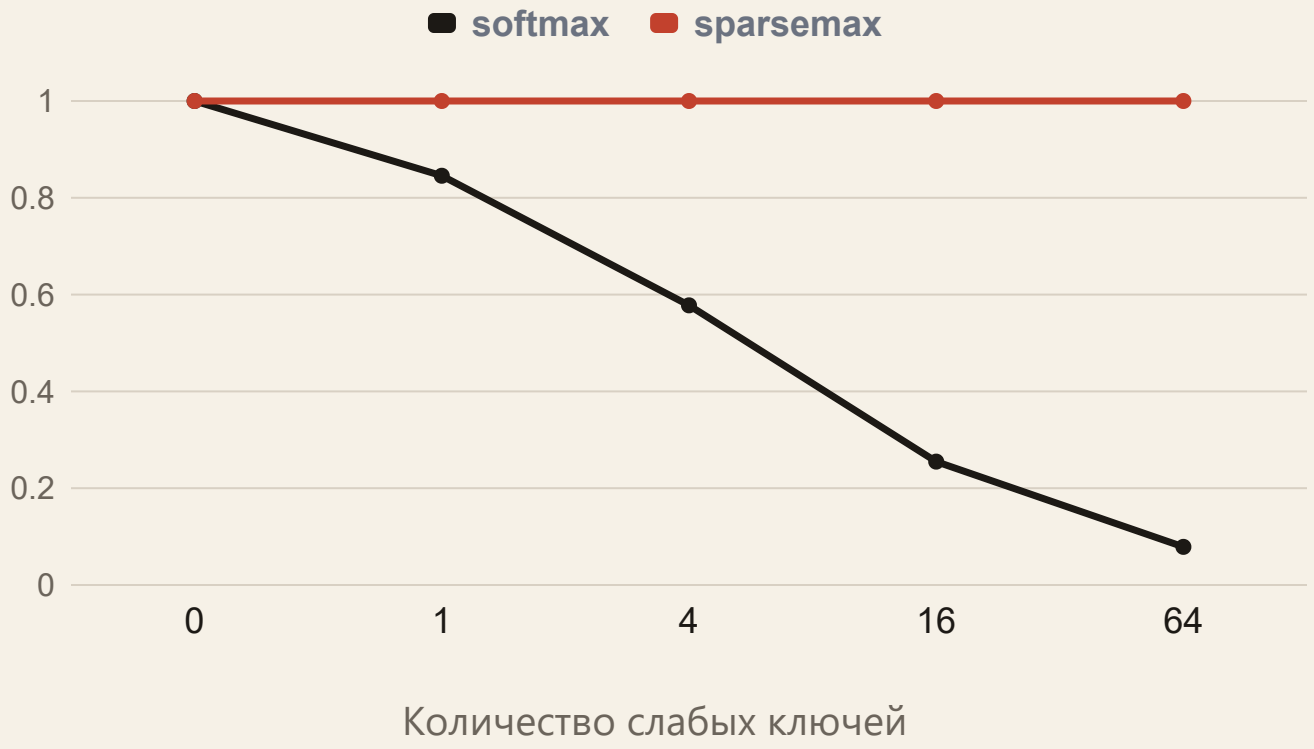
Что выбрал Миша? → **кофе**

Модель сравнивает запрос с ключами и смешивает связанные с ними значения. Для точного ответа полезно сосредоточить вес на нужной записи.

Ключ помогает найти запись. Значение несёт то, что нужно извлечь.

Слабые вклады накапливаются с длиной

Нужный ключ имеет скор 2,0. Каждый из N отвлекающих ключей имеет скор 0,3. Вертикальная ось: доля веса от 0 до 1.



Доля нужного ключа

$$1 / (1 + N \cdot e^{-1.7})$$

При 64 слабых ключах softmax оставляет нужному лишь 7,9% веса. Его скор при этом не изменился.

sparsemax отсекает эти ключи: 2,0 − 0,3 > 1.

Внимание можно прочитать как регрессию

Близкие ключи получают больший вес. Нормировка превращает эти веса в распределение.

$$p_i(q) = \frac{K_h(k_i - q)}{\Sigma_j K_h(k_j - q)} \qquad \hat{v}(q) = \Sigma_i p_i(q) v_i$$

q и k_i
Запрос и ключи в пространстве обучаемых признаков.

K_h и h
Ядро задаёт вес близости. h задаёт ширину окрестности.

v_i и $\hat{v}$
Значения и их взвешенное среднее. $\Sigma_i p_i = 1$.

Это оценка Надарая–Ватсона. Замена ядра меняет правило смешивания.

Softmax соответствует гауссову ядру

Для нормированных векторов большое скалярное произведение означает малое расстояние.



$$\tfrac{1}{2}\,\|k_i - q\|^2 = 1 - k_i^T q$$

$$K_h \propto \exp(-\|k_i - q\|^2 / 2h^2)$$

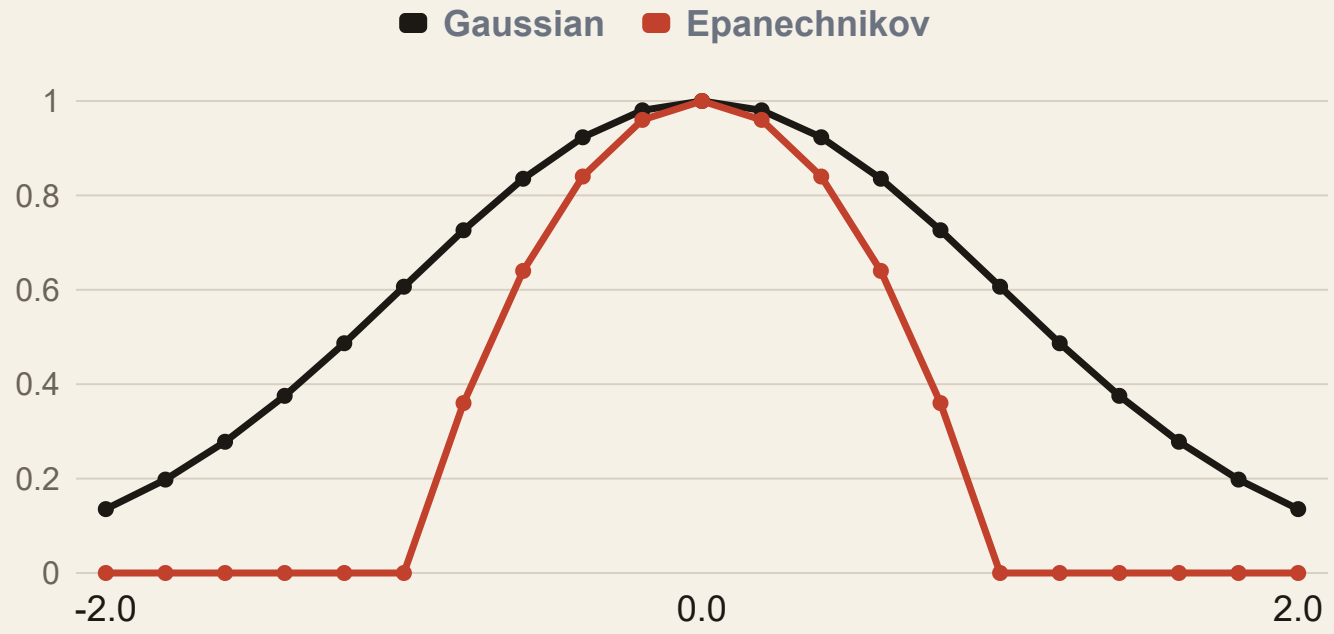
$$\mathbf{p = \mathrm{softmax}(z),\ z_i = k_i^T q / h^2}$$

Общая константа сокращается. Температура softmax равна h^2, ширина ядра равна h.

Дальше скор $z_i = k_i^T q / \gamma$. После деления на γ он может быть больше 1.

Компактное ядро имеет границу

Гауссово ядро $\exp(-u^2/2)$ имеет ненулевой хвост. Epanechnikov $[1-u^2]_+$ обнуляется при $|u| \geq 1$.



Смещение u от центра в единицах ширины

Компактный носитель

За границей окрестности ядро равно ровно нулю. Такие ключи совсем не участвуют в сумме, а число активных ключей может меняться.

Кривые приведены к пику 1. Общая константа сокращается при нормировке.

Sparsemax подбирает порог по скорам

Правило $p_i = [z_i - \tau]_+$. Порог τ выбирается так, чтобы сумма весов была равна 1.

Ключ	Скор z_i	$\max(z_i - 0{,}55, 0)$
A	1,2	0,65
B	0,9	0,35
C	0,5	0,00
D	0,2	0,00
E	-0,4	0,00

0,65 + 0,35 = 1. Дополнительного деления нет.

$$\tau = 0{,}55$$

$(1{,}2 - \tau) + (0{,}9 - \tau) = 1$

Отсюда $2{,}1 - 2\tau = 1$.

C, D и E лежат ниже порога. Их веса становятся нулевыми.

Одни скоры дают разные распределения

Темнее ячейка, больше её вес. normReLU сохраняет все положительные скоры, а entmax меняет форму отсечения.

Внимание	A: 1,2	B: 0,9	C: 0,5	D: 0,2	E: −0,4
softmax	35,6%	26,4%	17,7%	13,1%	7,2%
sparsemax	65,0%	35,0%	**0,0%**	**0,0%**	**0,0%**
1.5-entmax	50,7%	31,6%	13,1%	4,5%	**0,0%**
normReLU	42,9%	32,1%	17,9%	7,1%	**0,0%**

Точное обнуление и концентрация веса различаются даже при одинаковых скорах.

ReLUmax привязывает порог к лучшему ключу

Метод сохраняет все ключи в пределах заданного отступа от максимального скора. Число соседей заранее не фиксируется.

Ключ	z_i	$[z_i - m + \delta]_+$
A	1,2	0,8
B	0,9	0,5
C	0,5	0,1
D	0,2	0,0
E	-0,4	0,0

Лучший ключ всегда имеет положительный вес.

$$p_i \propto [z_i - m + \delta]_+^r$$

$m = \max z_i$, $\delta > 0$.
В примере $m = 1{,}2$, $\delta = 0{,}8$.

Порог $m - \delta = 0{,}4$.
Активны A, B и C.
После этого веса делятся на их сумму.

Статья объединяет механизмы одним семейством

При α-entmax $p_i = [(\alpha-1)z_i - \tau]_+^{1/(\alpha-1)}$, а τ обеспечивает $\Sigma_i p_i = 1$. Большая α усиливает разреженность.

$$K_h(u) \propto [1 - \|u\|^2/h^2]_+^r \quad \alpha = 1 + 1/r$$

r	Ядро	α	При адаптивной нормировке
1	Epanechnikov	2	sparsemax
2	Biweight	1,5	1.5-entmax
3	Triweight	4/3	4/3-entmax

Предел α → 1 даёт softmax. Одно ядро Epanechnikov даёт normReLU, sparsemax или ReLUmax в зависимости от выбора порога и нормировки.

Порог и число соседей задают разные ограничения

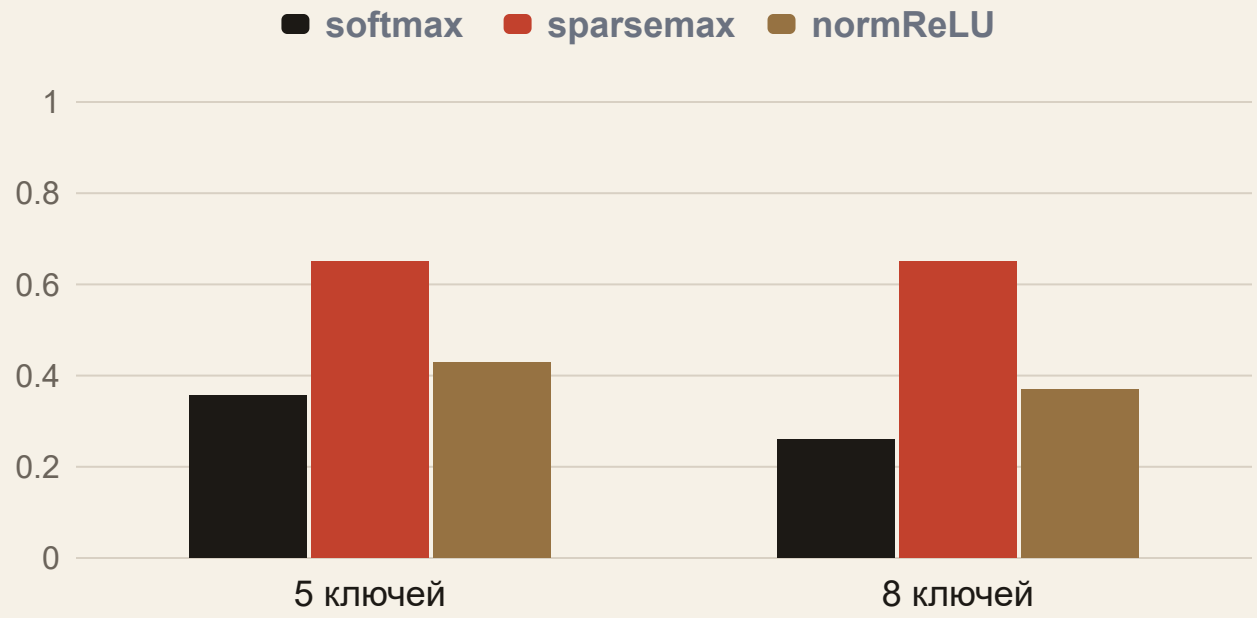
Носитель — набор ключей с ненулевыми весами. У sparsemax, entmax и ReLUmax его размер подстраивается под текущие скоры.

Механизм	Кого сохраняет	Что заранее фиксировано
normReLU	Все ключи со скором $z_i > 0$	Порог 0
sparsemax / entmax	Выше адаптивного порога	Форма ядра, параметр α
ReLUmax	Ключи в пределах δ от max z	Отступ δ и степень r
top-k softmax	k ключей с наибольшими скорами	Число ключей k

top-k применяет softmax внутри выбранного набора. uniform kNN даёт каждому из k соседей вес 1/k.

Фиксированный порог ноль ограничивает normReLU

Добавим три слабых ключа со скором 0,15. Они ниже порога sparsemax, но положительны для normReLU.



Доля A: 35,6% → 25,9% / 65% → 65% / 42,9% → 36,9%

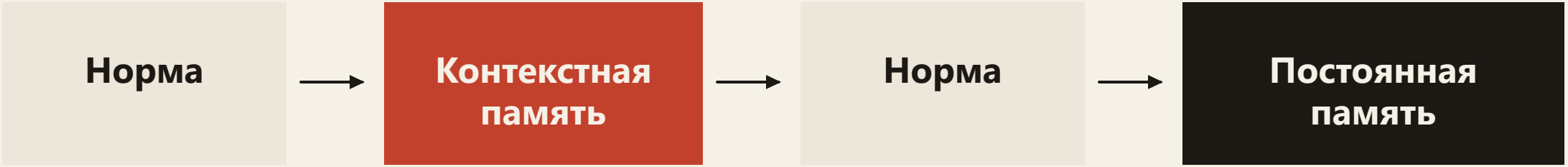
normReLU(cz) = normReLU(z), c > 0

Положительный масштаб сокращается. Ключи с $k_i^T q > 0$ остаются в носителе, поэтому температура не сужает его. У sparsemax масштаб влияет на отбор.

Все скоры ≤ 0 дают 0/0. В статье и воспроизведении это заменяют равномерными весами.

Эксперименты меняют ядро в Memory Mosaics

Это архитектура, в которой внимание явно реализует ядерную регрессию. Нормы и две памяти образуют повторяемый блок.



Меняется только контекстная память
Запрос q_t совпадает с ключом k_t. Ключи усредняют прошлое. Значение v_i включает следующий токен.

Обучаемые слоты сохраняют softmax
Они хранят общие знания между примерами. Позиционных эмбеддингов нет, но усреднение ключей учитывает порядок.

Читаем лишь i < t: значение v_{t-1} знает x_t, но будущий x_{t+1} в ответ не попадает.

Длина проверяет перенос выученного правила

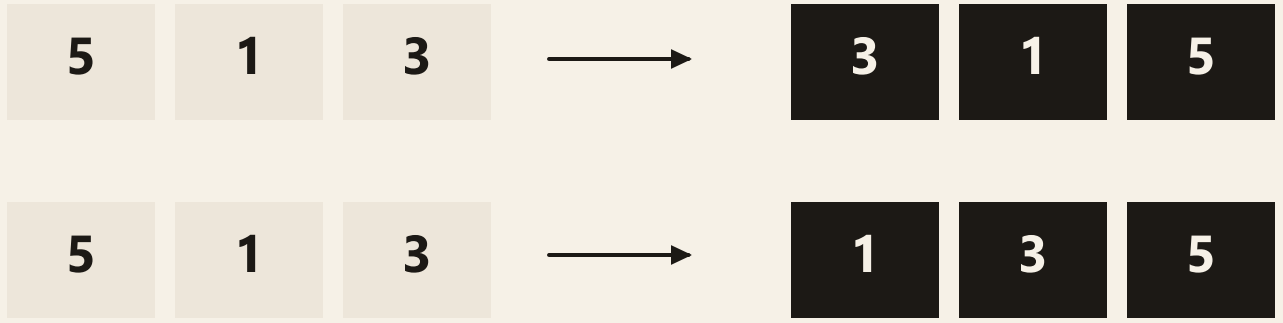
Модель учится на содержательной длине до 64. Затем получает больше записей или более длинный список. 1× = 64, 2× = 128, 64× = 4096.

MQMTAR

Извлечение пары
по ключу из 2 токенов

[7 2] : [5 9] [4 1] : [8 3] ...

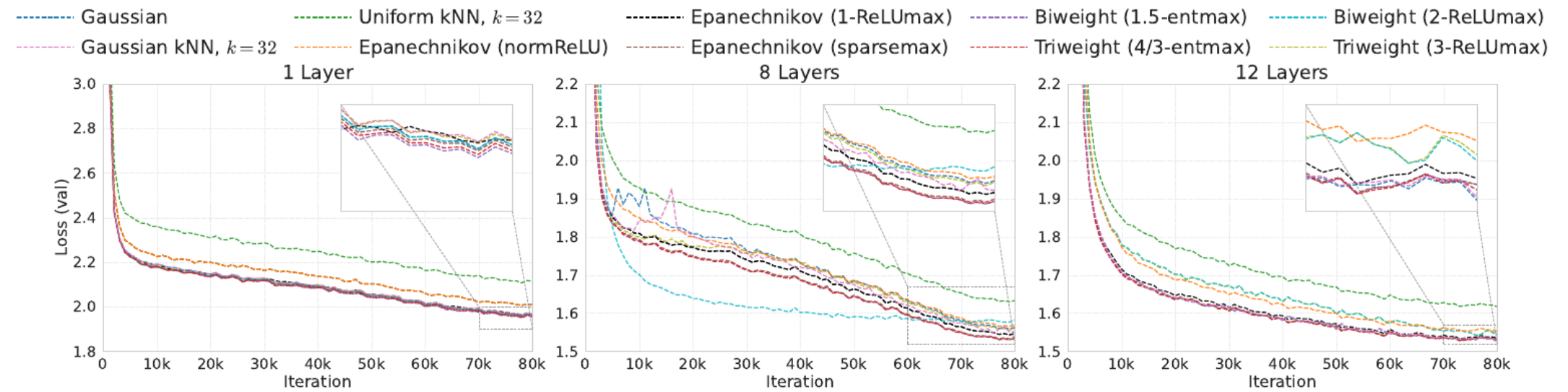
Запрос [4 1] Ответ [8 3]



Exact match: весь ответ верен. Для MQMTAR нужны 4 пары ответов, то есть все 8 токенов.

На BabiStories преимущество зависит от глубины

Языковая модель предсказывает следующий токен в простых историях. Ниже loss лучше. Контекст 512, ширина 768, обучение 80 тыс. шагов.



1 и 8 слоёв

Адаптивные компактные ядра дают меньший validation loss.

12 слоёв

Разрыв с softmax сужается. normReLU и uniform kNN отстают.

На RegBench адаптивные ядра конкурентны

RegBench проверяет обучение правила в контексте. Здесь показан режим с малым числом данных. TVD = ½Σ |p − p*|, ниже лучше.

Контекст: строки одного случайного автомата

ab / aab / abb / aa...?

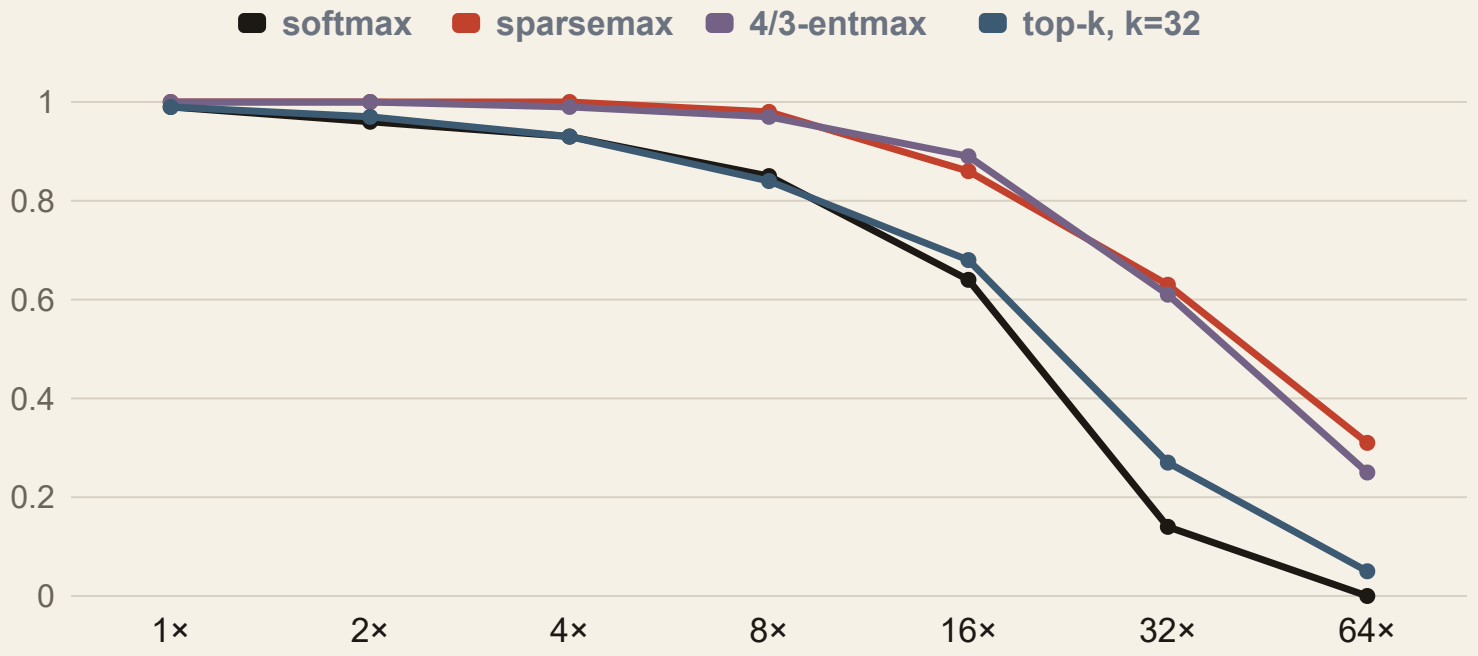
Модель выводит правило генерации из примеров. Тест содержит новые автоматы. Accuracy оценивает последний токен, TVD сравнивает распределения.

Ядро	Accuracy ↑	TVD ↓
softmax	93,9%	34,1%
sparsemax	94,2%	32,5%
1.5-entmax	93,8%	33,0%
normReLU	84,3%	43,6%

При 1 тыс. обучающих примеров разница с softmax мала. normReLU заметно хуже.

На длинном MQMTAR sparsemax сохраняет точность

Адаптивные компактные ядра лучше сохраняют извлечение при росте числа записей. Даже у них точность снижается к 64×.



Exact match от 0 до 1. Каждое удвоение длины имеет равный шаг по оси.

31% против 0%

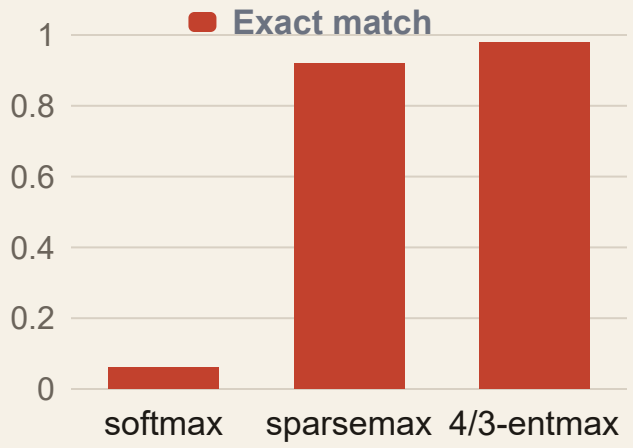
На длине 4096:
sparsemax и softmax.
Среднее по 3 сидам.

normReLU на 64×:
среднее 0,31,
лучший сид 0,94.
Результат нестабилен.

Лучшее ядро зависит от задачи

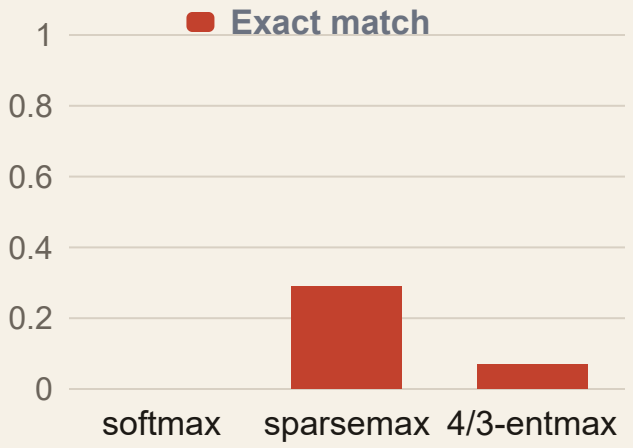
Sparsemax помогает сортировке, но на Reverse 96 уступает softmax. Одной разреженности недостаточно для универсального выигрыша.

Sort, 128



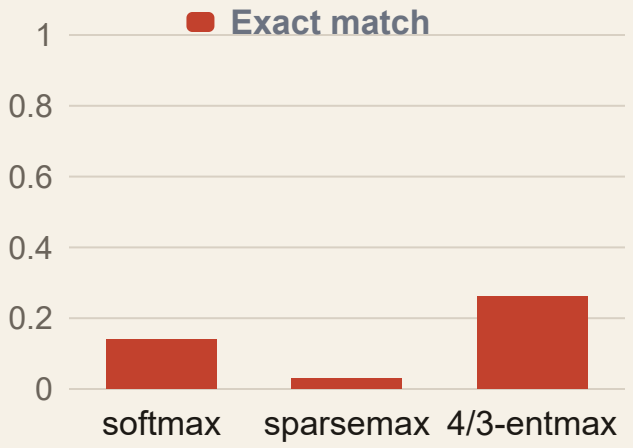
0,06 / 0,92 / 0,98

Sort, 256



0,00 / 0,29 / 0,07

Reverse, 96



0,14 / 0,03 / 0,26

Среди всех методов: Sort 256 лучше у 2-ReLUmax (0,34). Reverse 96 лучше у 3-ReLUmax (0,29).

В воспроизведении модель и бюджет меньше, чем в статье

Сравнение ядер внутри одного прогона полезно. Оценки статьи и воспроизведения получены при разных протоколах.

Параметр	Статья	Воспроизведение
Ширина / слоты	512 / 2688	256 / 512
Примеры MQMTAR	50 млн	10,24 млн
Примеры Reverse / Sort	30 / 40 млн	0,512 / 0,512 млн
Сиды / тест MQMTAR	3 / до 4096	1 / до 1024
top-k для этих задач	32	8

Совпадают глубина 4 / 4 / 2, 8 голов и длина ≤ 64.

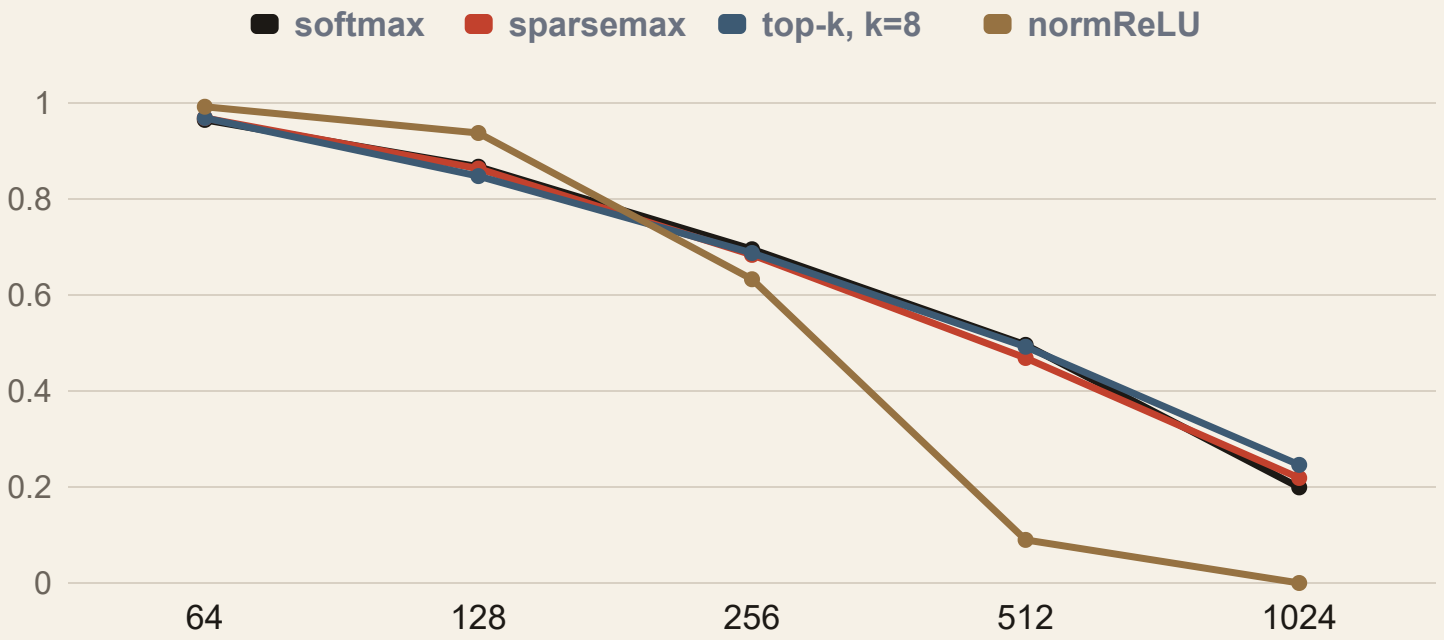
RTX 5070, 12 ГБ. Модель и обучение уменьшены.

Проверена часть эффектов статьи.

Тест MQMTAR: 256 примеров на длину. Один ответ меняет оценку примерно на 0,39 п.п.

Главный разрыв на MQMTAR пока не воспроизведён

Exact match от 0 до 1. На обучающей длине все ядра выше 0,96. При росте длины точность падает у всех шести вариантов.



К длине 1024 все варианты, кроме normReLU, лежат в диапазоне 17,58–24,61%.

На длине 1024

softmax: 19,92%
sparsemax: 21,88%
top-k: 24,61%
normReLU: 0%

Большой разрыв из статьи здесь не получен. Разница sparsemax и softmax мала.

Результат чувствителен к режиму обучения

Архитектура одинакова. Первый прогон: батч 32 и 20 тыс. шагов. Второй: батч 256 и 40 тыс. шагов. Оба используют исправленный entmax.

Ядро	640 тыс. примеров	10,24 млн примеров	Изменение, п.п.
softmax	2,34%	19,92%	+17,58
sparsemax	1,95%	21,88%	+19,93
top-k	20,31%	24,61%	+4,30
4/3-entmax	15,62%	23,44%	+7,82

Большее обучение повышает оценки, но одновременно меняются батч и число шагов. Эффект ядра от эффекта протокола здесь не изолирован.

На Sort 128 преимущество заметно, на 256 исчезает

Все значения: exact match. На Reverse 96 лучший локальный результат у 1.5-entmax. 4/3-entmax слабо обучился Sort уже на длине 64.

Ядро	Reverse 64	Reverse 96	Sort 64	Sort 128	Sort 256
softmax	98,8%	57,8%	90,6%	18,4%	0,0%
sparsemax	99,6%	49,6%	94,5%	84,4%	0,8%
normReLU	95,7%	0,4%	83,6%	0,0%	0,0%
top-k	99,6%	53,9%	82,0%	16,4%	0,0%
1.5-entmax	100,0%	69,1%	95,7%	85,9%	0,0%
4/3-entmax	99,2%	60,6%	55,5%	24,2%	0,0%

Sort 128: sparsemax и 1.5-entmax около 85%. На Sort 256 все ниже 1%.

Локальный RegBench показывает небольшой разрыв

sparsemax имеет наибольшую accuracy, normReLU наименьшую. Генератор и режим обучения отличаются от оригинального бенчмарка.

Ядро	Accuracy ↑	TVD ↓
softmax	44,92%	48,35%
sparsemax	48,05%	47,98%
normReLU	40,62%	48,97%
top-k	45,31%	47,38%
1.5-entmax	44,14%	47,11%
4/3-entmax	46,09%	47,88%

Упрощённый генератор
2000 обучающих автоматов, 256 тестовых. 8–12 строк длиной 2–6. Обучение: 4000 шагов.

Определение локальных метрик
Accuracy сравнивает с конкретным случайным символом. TVD сравнивает с распределением разрешённых переходов. Из-за другого протокола числа несопоставимы с Table 1.

Воспроизведение подтверждает часть эффектов

Теоретическое соответствие проверяется математически. Воспроизведение проверяет, сохраняется ли практический выигрыш при меньших ресурсах.

Наблюдение в воспроизведении	Что из него следует
Sort 128: около 85% у sparsemax / 1.5-entmax	Перенос на 2× длину лучше, чем у softmax
MQMTAR 1024: 17,58–24,61% у пяти ядер	Сильное разделение методов из статьи не повторилось
normReLU падает на длинных синтетических задачах	Согласуется с ограничением порога 0
Один сид и разные бюджеты обучения	Устойчивость порядка методов неизвестна

Дальше: несколько сидов, одинаковый протокол, больший бюджет.

Масштабирование и скорость остаются открытыми

Статья исследует качество моделей и предлагает теорию разреженности. Представленные результаты не дают общего обещания ускорения или выигрыша на любой задаче.

Границы самой статьи

Эксперименты используют Memory Mosaics и модели меньше 1 млрд параметров.

Языковой контекст равен 512. Есть отдельный OOD-тест на Simple English Wikipedia, но перенос на большие LLM ещё требует проверки.

Что не вошло в воспроизведение

Не запускались BabiStories и Wikipedia OOD.

Нет ReLUmax, uniform kNN, длины MQMTAR 4096 и трёх сидов.

В воспроизведении строится плотная матрица скоров: нулевые веса сами по себе не ускоряют расчёт. В статье скорость зависит от реализации, включая AdaSplash.

Все шесть ядер в основном прогоне MQMTAR

Exact match в процентах. Обучение: 10,24 млн примеров, 4 слоя, ширина 256, 512 слотов памяти, батч 256.

Ядро	64	128	256	512	1024
softmax	96,48%	86,72%	69,53%	49,61%	19,92%
sparsemax	96,88%	86,33%	68,36%	46,88%	21,88%
normReLU	99,22%	93,75%	63,28%	8,98%	0,00%
top-k	96,88%	84,77%	68,75%	49,22%	24,61%
1.5-entmax	96,48%	85,94%	65,62%	47,66%	17,58%
4/3-entmax	97,66%	92,58%	74,61%	49,61%	23,44%

Показаны исходные оценки. Точность до сотых процента не означает такую же статистическую уверенность.

Выбор ядра меняет способ извлечения из памяти

Sparsemax и entmax существовали раньше. Вклад статьи: их общее объяснение через регрессию, новое ReLUmax и сравнительные эксперименты.